

COS 433: Cryptography

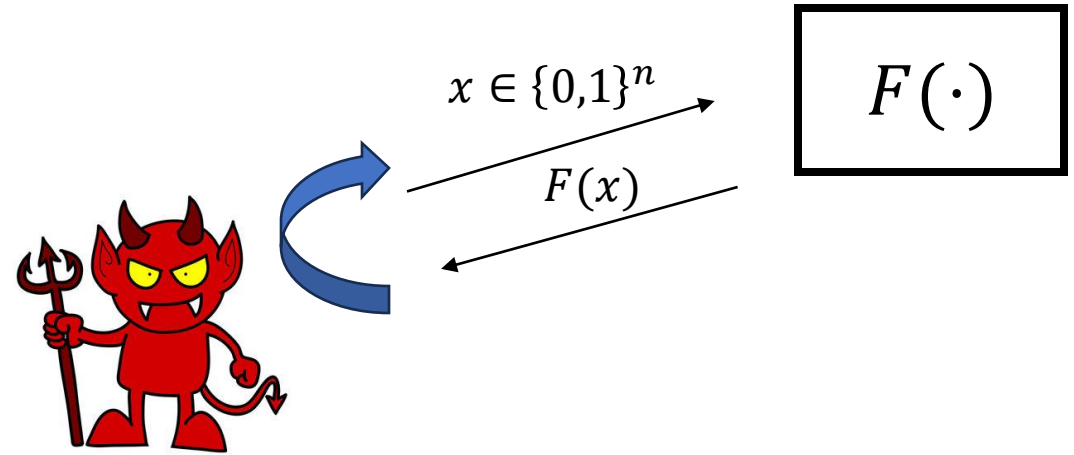
Reminders:

- Problem Set 1 due 9/24
- Problem Set 2 to be released on 9/24

Last time: PRFs

Pseudorandom Functions (PRFs)

A poly-time **oracle algorithm** $A^{F(\cdot)}(1^\lambda)$ is an algorithm that can call a function $F(\cdot)$ on inputs of length $\text{poly}(\lambda)$ **at unit cost**



Definition: An efficiently computable function family $\text{PRF}: \{0,1\}^\lambda \times \{0,1\}^n \rightarrow \{0,1\}^m$ is a PRF family if:

$$\text{For all polynomial-time } A^{F(\cdot)}, A^{\text{PRF}(\textcolor{red}{s}, \cdot)}(1^\lambda) \approx_c A^{\textcolor{green}{F}(\cdot)}(1^\lambda)$$

Building PRFs

Theorem (GGM86): if PRGs exist, then so do **PRFs!**

Goldreich

–

Goldwasser

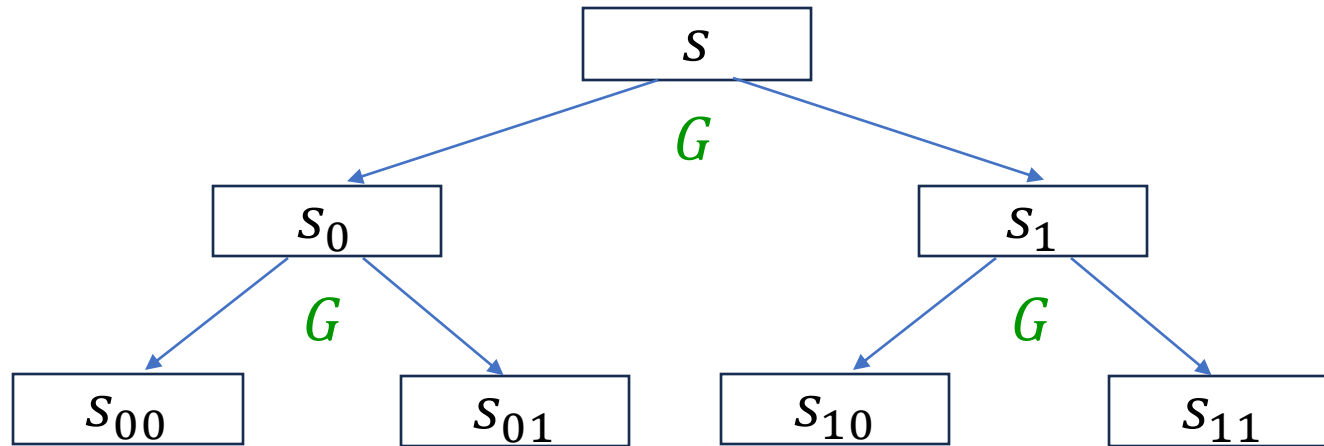
–

Micali



Building PRFs

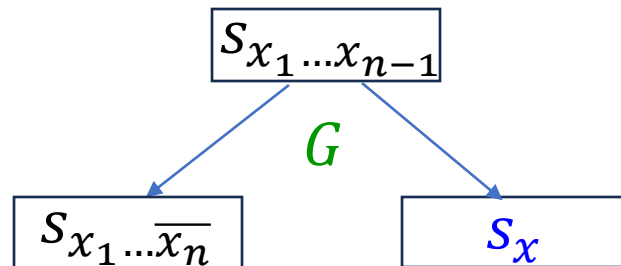
Let $G: \{0,1\}^\lambda \rightarrow \{0,1\}^{2\lambda}$ denote a **length doubling PRG**



Layer i : 2^i vertices

...

(suppose $x_n = 1$)



Define $\text{PRF}_s(x) = s_x$

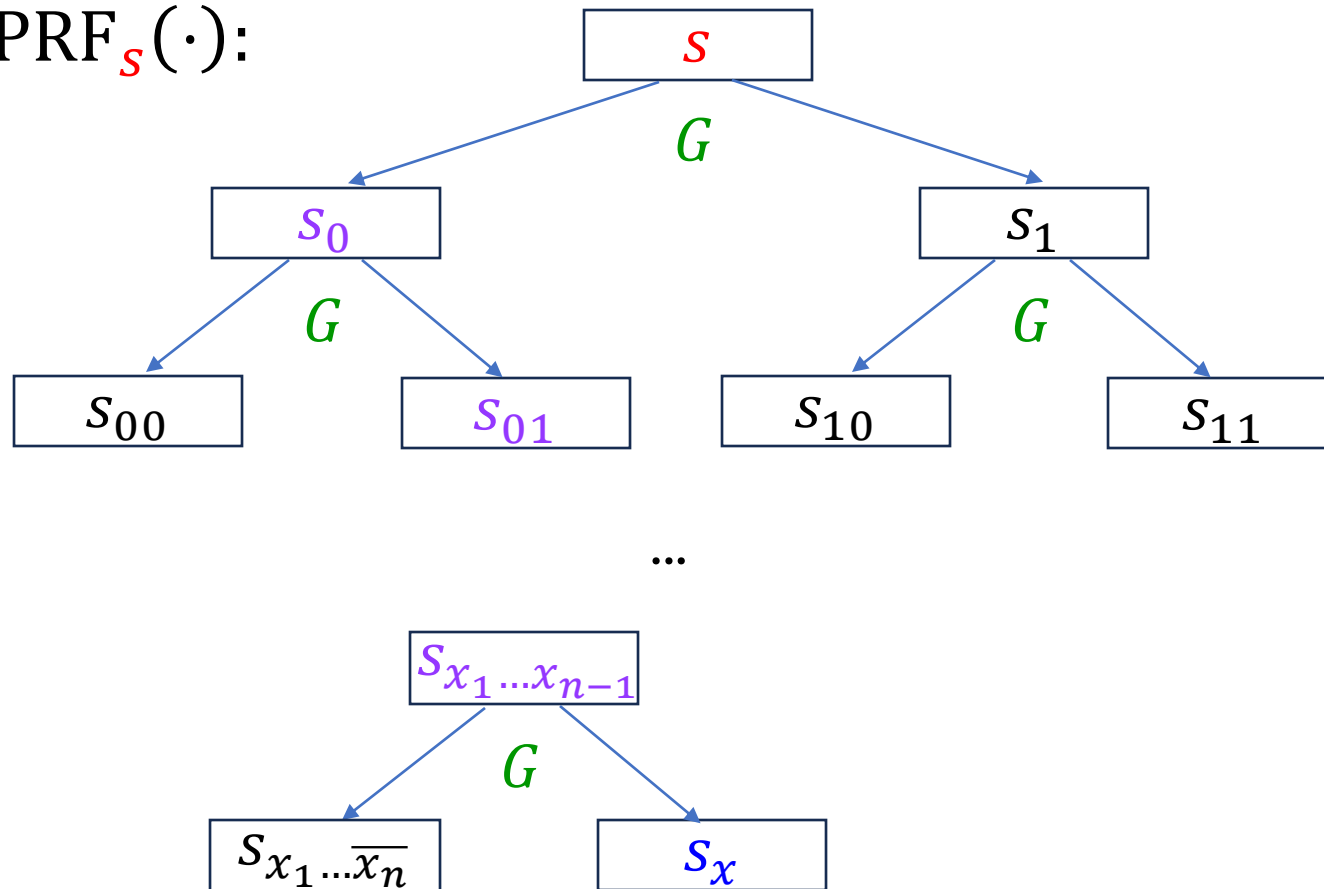


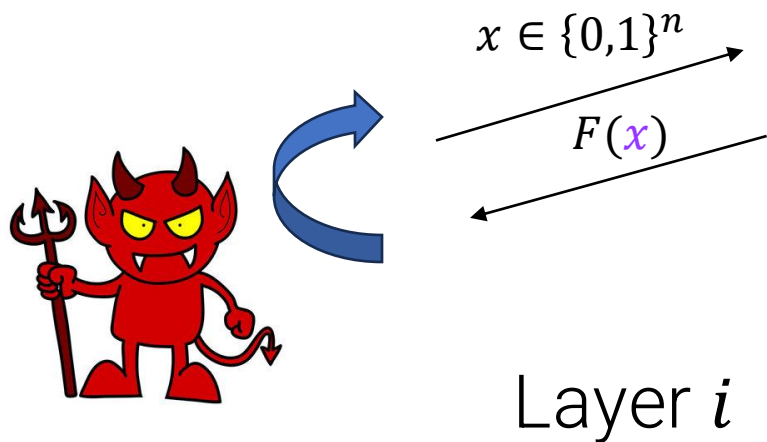
$x \in \{0,1\}^n$

$F(x)$

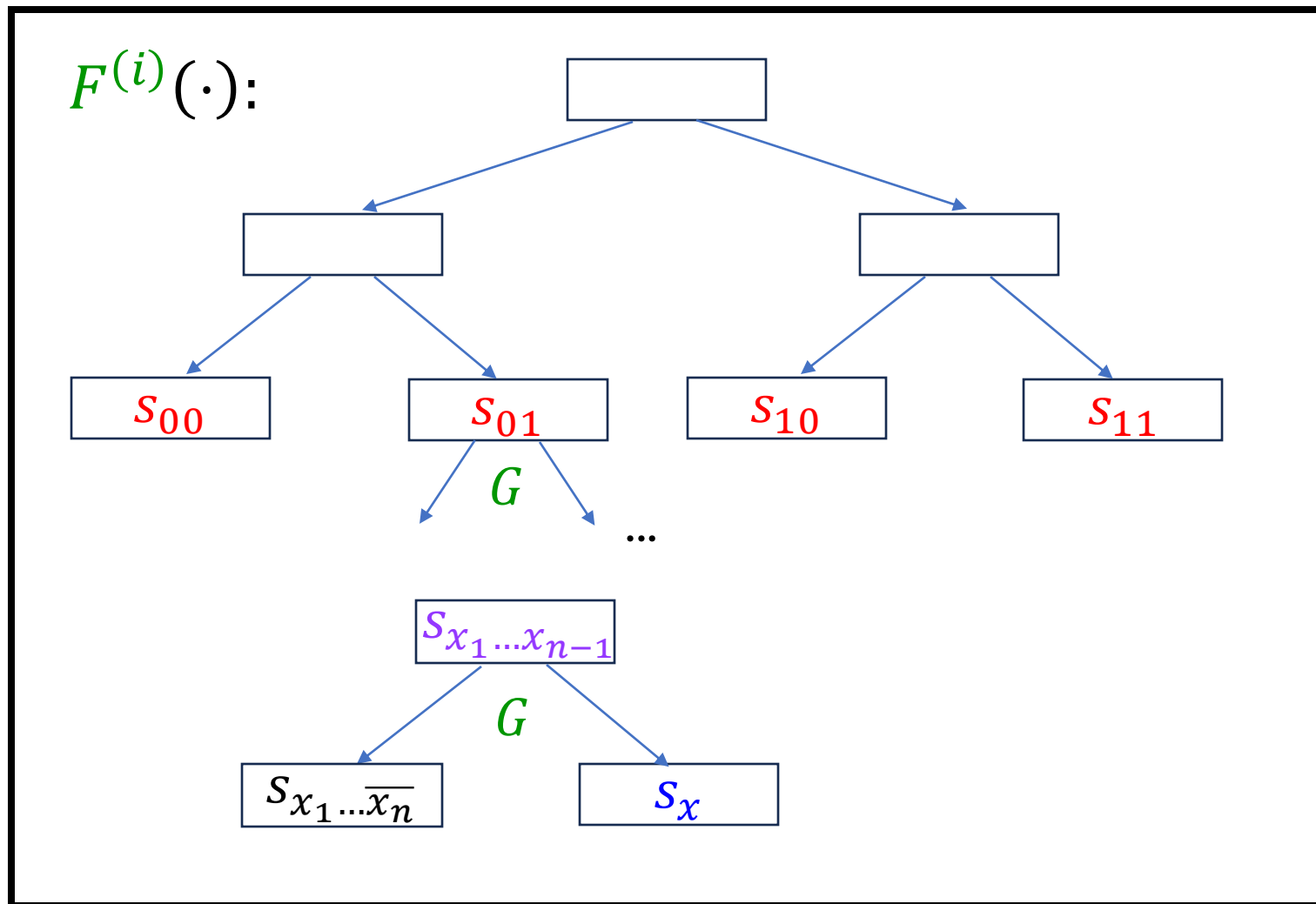
How do we prove
pseudorandomness?

$\text{PRF}_s(\cdot)$:





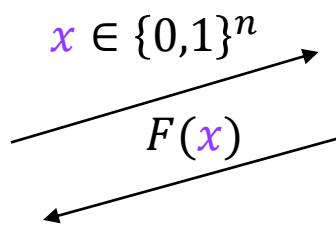
Function distribution $F^{(i)}$



- $F^{(i)}(x)$ applies a uniformly random function to $x_1 \dots x_i \rightarrow s_{x_1 \dots x_i}$
- Computes s_x from $s_{x_1 \dots x_i}$ using G .

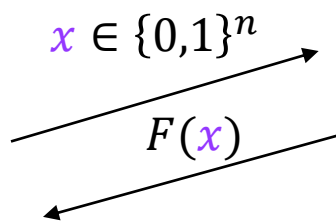
Need:  $F^{(i)}(\cdot)$ $\approx_c$  $F^{(i+1)}(\cdot)$

- We will use **another hybrid argument!**
- Worry: 2^i different seeds in $F^{(i)}$ -- can't change them one by one...
- It's ok: adversary only accesses poly many of them.



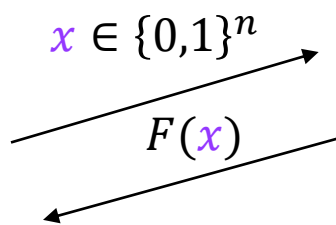
$F^{(i)}(\cdot)$:

- Maintain a list of $(x_1 \dots x_i, y)$
- If query is on the list, output $\text{PRF}_y(x_{i+1} \dots x_n)$
- If not, sample y and add (x, y) to list



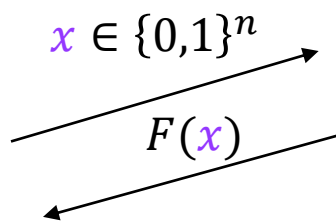
$F^{(i+1)}(\cdot)$:

- Maintain a list of $(x_1 \dots x_i, y_0, y_1)$
- If query is on the list, output $\text{PRF}_{y_{x_{i+1}}}(x_{i+2} \dots x_n)$
- If not, sample y_0, y_1 and add (x, y_0, y_1) to list



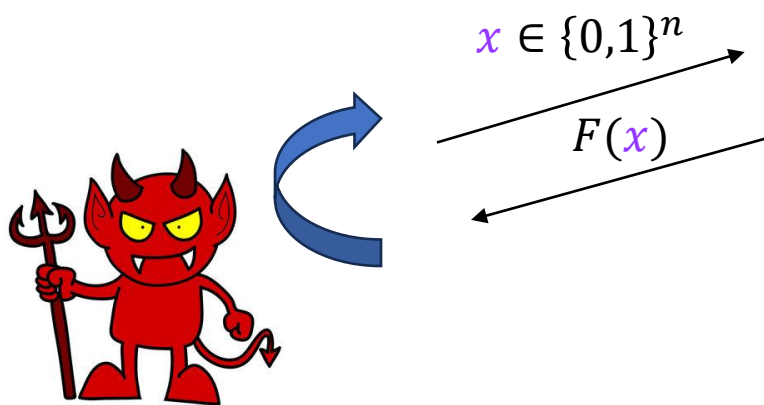
$F^{(i)}(\cdot)$:

- Maintain a list of $(x_1 \dots x_i, y_0, y_1)$
- If query is on the list, output $\text{PRF}_{y_{x_{i+1}}}(x_{i+2} \dots x_n)$
- If not, sample y and add $(x, G(y))$ to list



$F^{(i+1)}(\cdot)$:

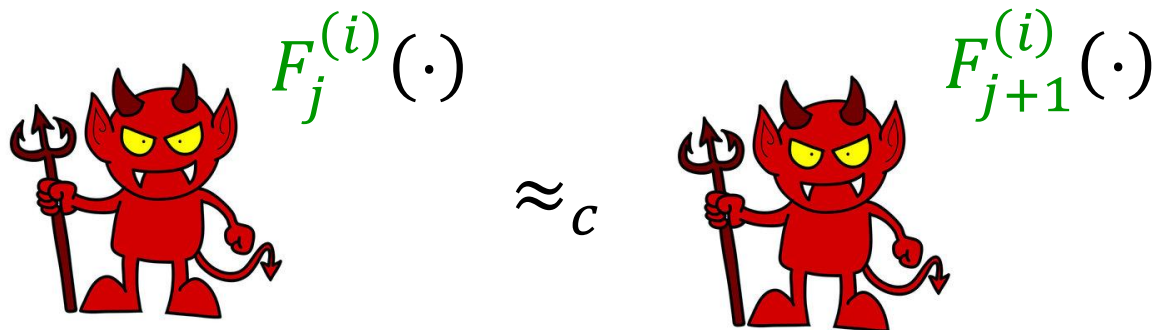
- Maintain a list of $(x_1 \dots x_i, y_0, y_1)$
- If query is on the list, output $\text{PRF}_{y_{x_{i+1}}}(x_{i+2} \dots x_n)$
- If not, sample y_0, y_1 and add (x, y_0, y_1) to list

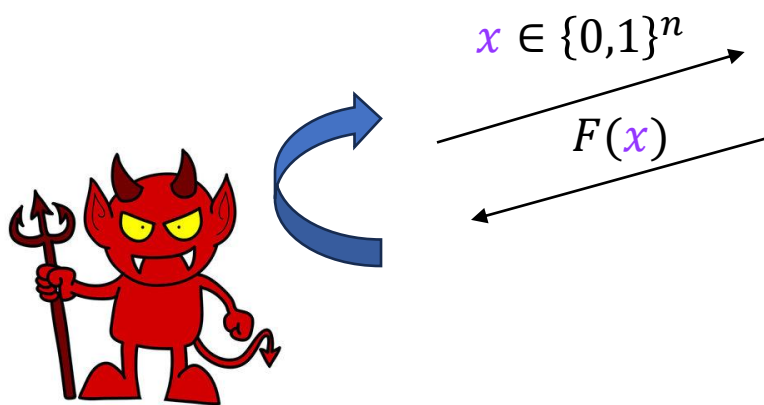


$F_j^{(i)}(\cdot)$:

- Maintain a list of $(x_1 \dots x_i, y_0, y_1)$
- If query is on the list, output $\text{PRF}_{y_{x_{i+1}}}(x_{i+2} \dots x_n)$
- If not:
 - (First j times) sample y and add $(x, G(y))$
 - (Otherwise) sample y_0, y_1 and add (x, y_0, y_1)

PRG security implies:

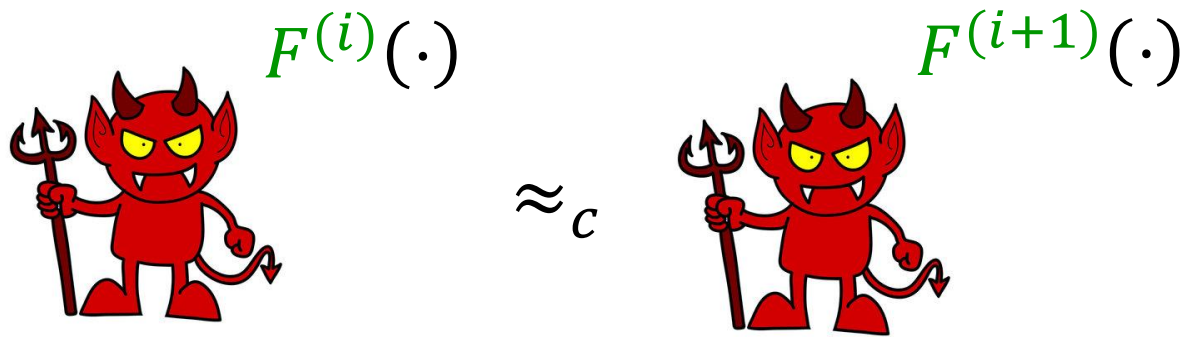


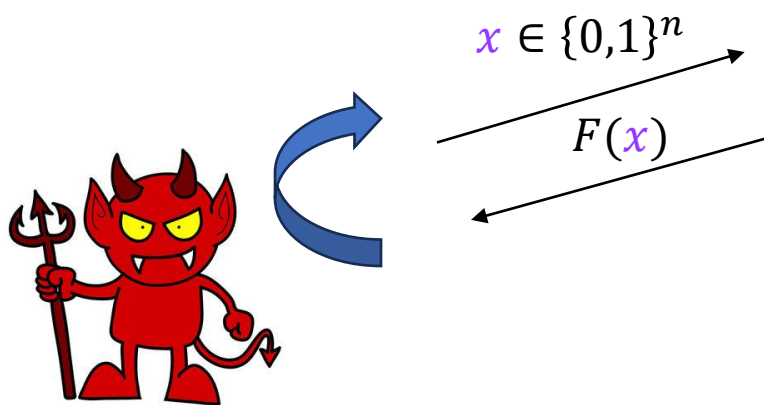


$F_j^{(i)}(\cdot)$:

- Maintain a list of $(x_1 \dots x_i, y_0, y_1)$
- If query is on the list, output $\text{PRF}_{y_{x_{i+1}}}(x_{i+2} \dots x_n)$
- If not:
 - (First j times) sample y and add $(x, G(y))$
 - (Otherwise) sample y_0, y_1 and add (x, y_0, y_1)

Therefore:

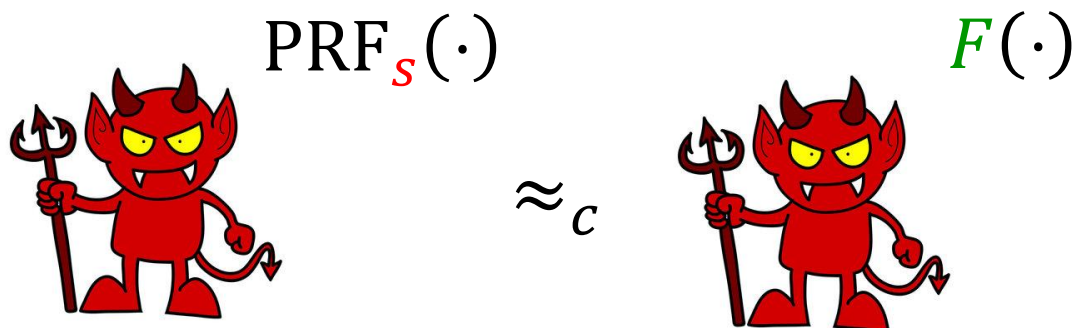




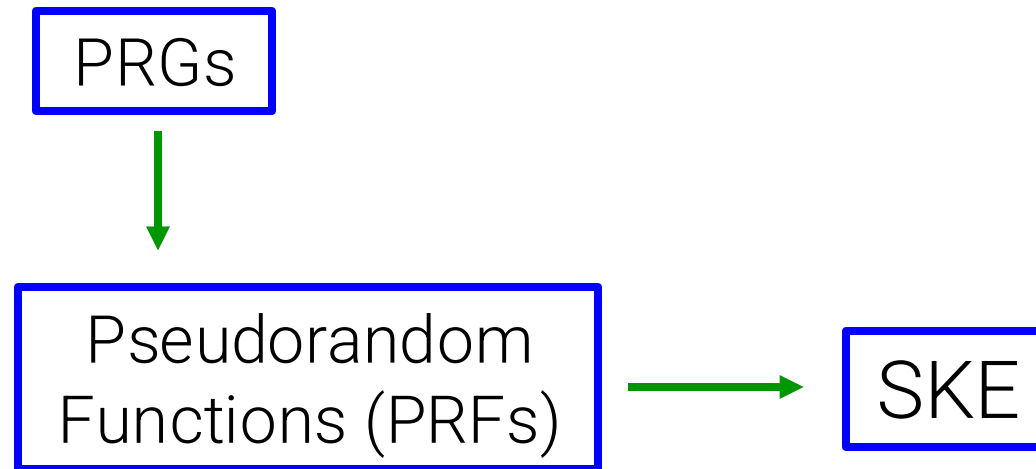
$F_j^{(i)}(\cdot)$:

- Maintain a list of $(x_1 \dots x_i, y_0, y_1)$
- If query is on the list, output $\text{PRF}_{y_{x_{i+1}}}(x_{i+2} \dots x_n)$
- If not:
 - (First j times) sample y and add $(x, G(y))$
 - (Otherwise) sample y_0, y_1 and add (x, y_0, y_1)

Finally:



Summary



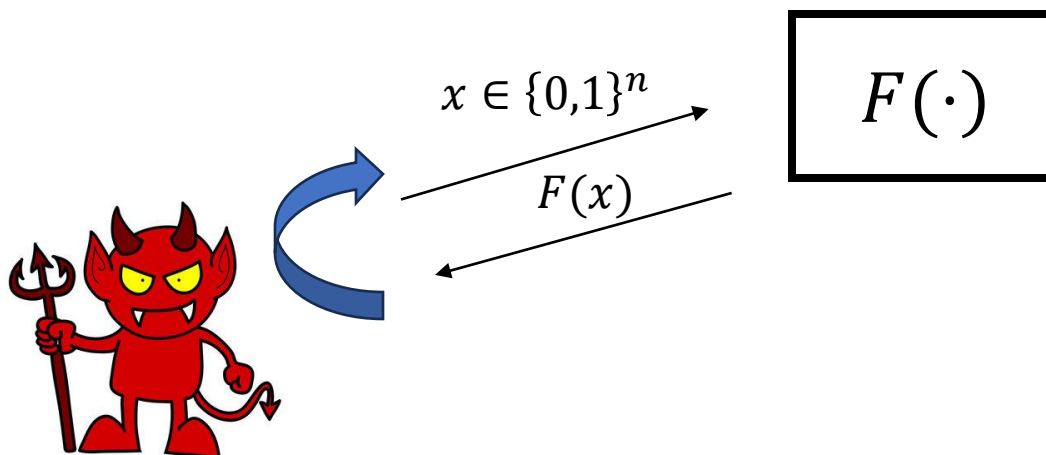
Next: what do “practical” SKE schemes look like?

Pseudorandom Permutations/Block Ciphers

Pseudorandom Permutations (PRPs)

Definition: An efficiently computable function family $P: \{0,1\}^\lambda \times \{0,1\}^n \rightarrow \{0,1\}^n$ is a PRP family if:

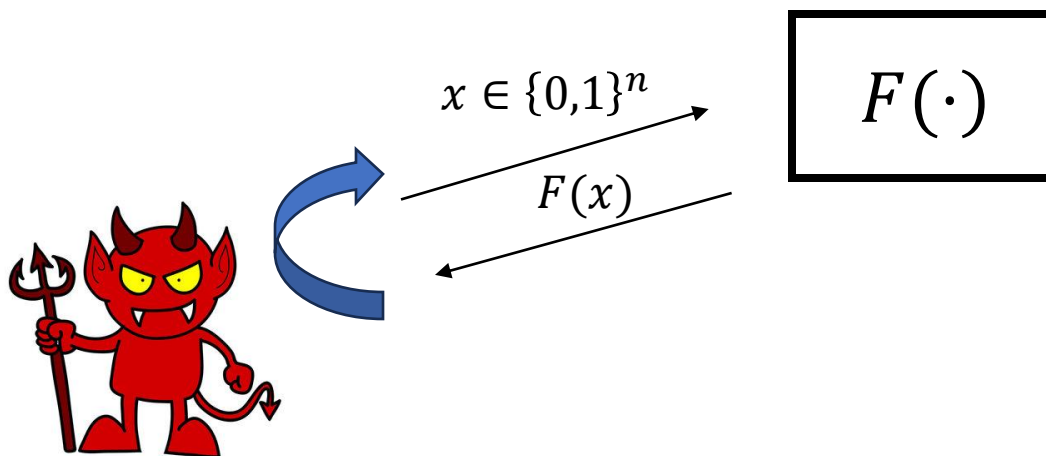
1. **Invertibility:** there is an efficiently computable inverse map $P^{-1}(s, \cdot)$ such that $P^{-1}(s, P(s, x)) = x$ for all x .
2. For all polynomial-time $A^{F(\cdot)}$, $A^{P(s, \cdot)}(1^\lambda) \approx_c A^{P(\cdot)}(1^\lambda)$



Pseudorandom Permutations (PRPs)

Definition 2: An efficiently computable function family $P: \{0,1\}^\lambda \times \{0,1\}^n \rightarrow \{0,1\}^n$ is a PRP family if:

1. **Invertibility:** there is an efficiently computable inverse map $P^{-1}(s, \cdot)$ such that $P^{-1}(s, P(s, x)) = x$ for all x .
2. For all polynomial-time $A^{F(\cdot)}$, $A^{P(s, \cdot)}(1^\lambda) \approx_c A^{F(\cdot)}(1^\lambda)$ (random function)

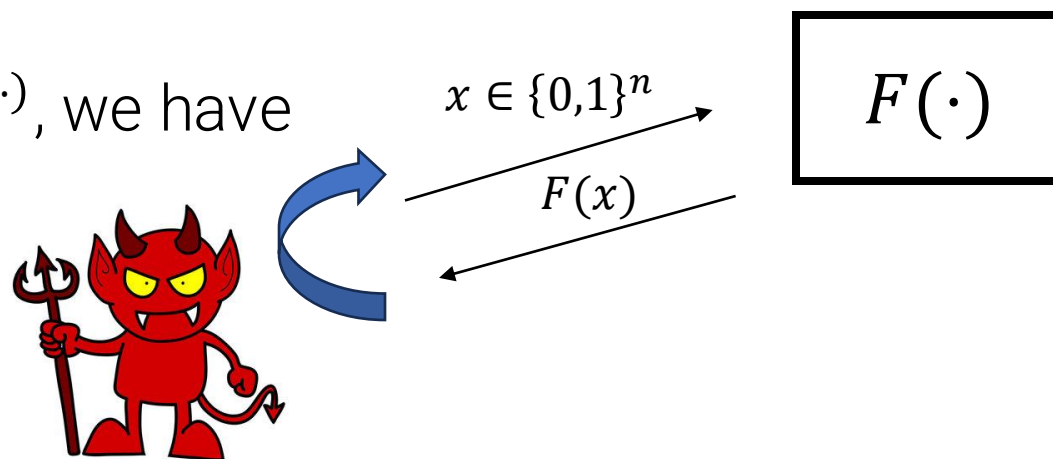


Pseudorandom Permutations (PRPs)

Theorem: Definition 1 is equivalent to Definition 2

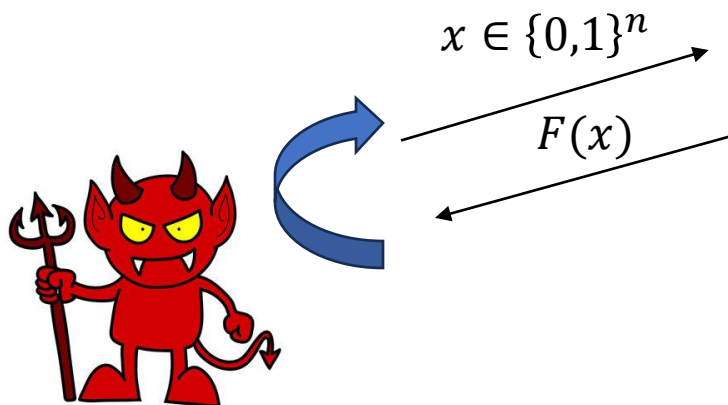
Proof: the claim is that for all polynomial-time $A^{F(\cdot)}$, we have

$$A^{P(\cdot)}(1^\lambda) \approx_c A^{F(\cdot)}(1^\lambda)$$



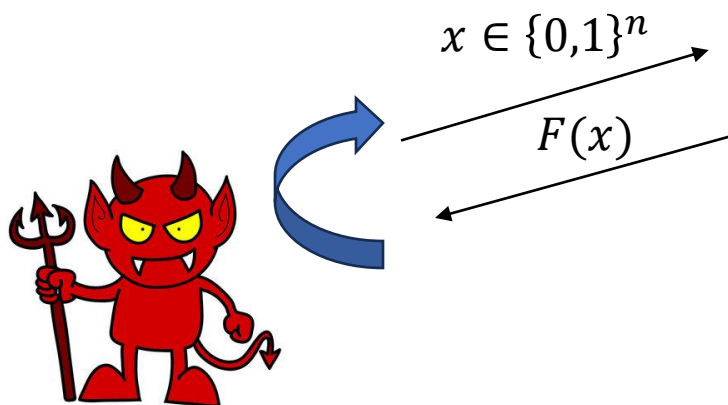
In fact: claim is true even **for inefficient** $A^{F(\cdot)}$, as long as A only calls $F(\cdot)$ **polynomially many times**! (“query-bounded” adversaries)

Intuition: if you query $F(\cdot)$ poly many times, highly unlikely to encounter collision



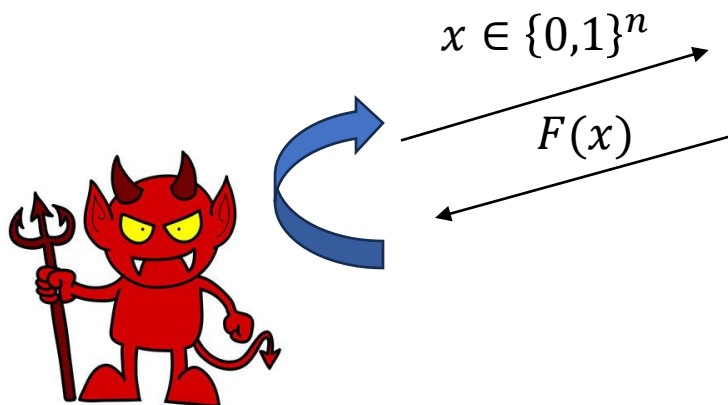
$F(\cdot)$:

- Maintain a list of (x, y)
- If query is on the list, output y
- If not, sample y and add (x, y) to list



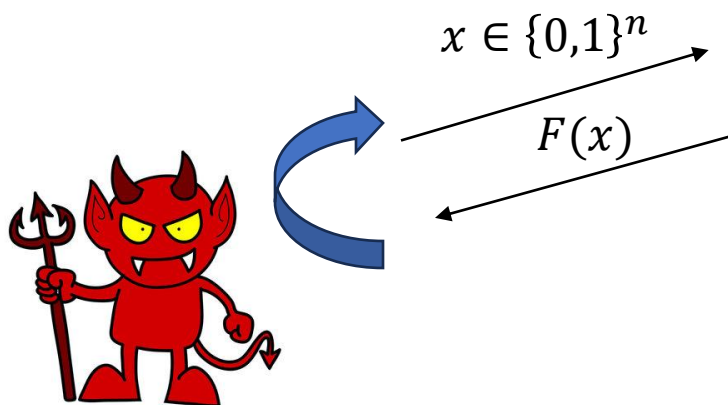
$P(\cdot)$:

- Maintain a list of (x_j, y_j) . Let $S = \{y_j\}$
- If query is on the list, output y
- If not, sample $y \notin S$ and add (x, y) to list



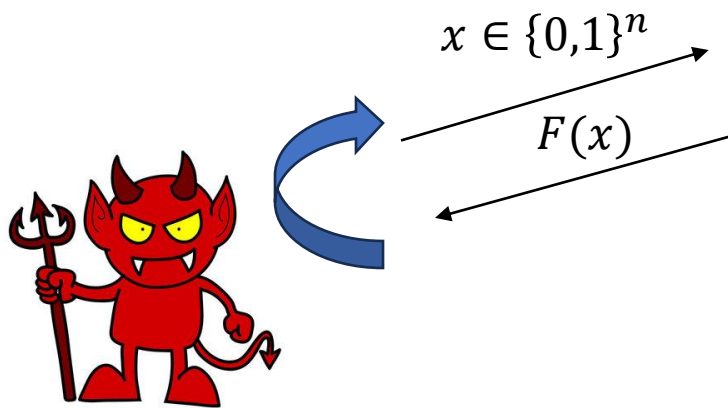
$F^{(i)}(\cdot)$:

- Maintain a list of (x_j, y_j) . Let $S = \{y_j\}$
- If query is on the list, output y
- If not,
 - i times: sample y and add (x, y) to list
 - after: sample $y \notin S$ and add (x, y) to list



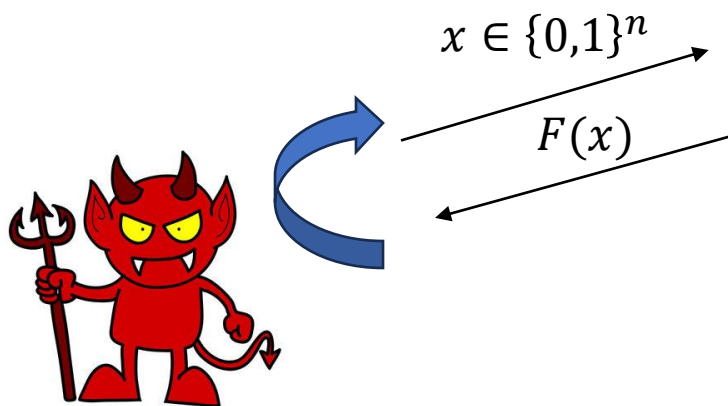
$P(\cdot)$:

- Maintain a list of (x_j, y_j) . Let $S = \{y_j\}$
- If query is on the list, output y
- If not, sample $y \notin S$ and add (x, y) to list



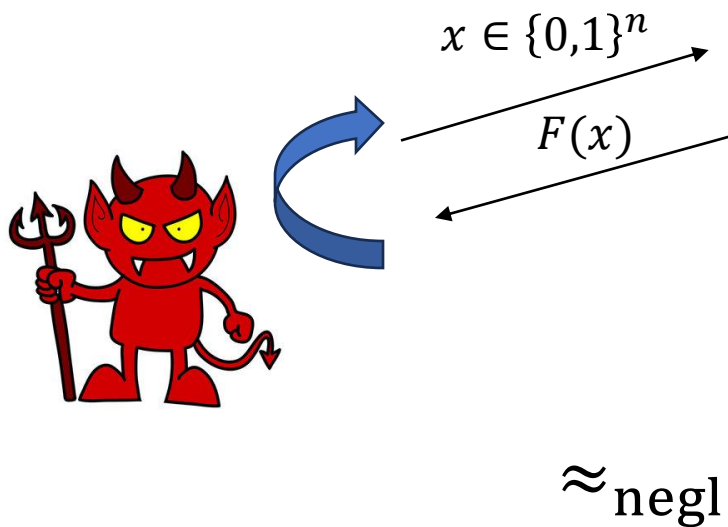
$F^{(i)}(\cdot)$:

- Maintain a list of (x_j, y_j) . Let $S = \{y_j\}$
- If query is on the list, output y
- If not,
 - i times: sample y and add (x, y) to list
 - after: sample $y \notin S$ and add (x, y) to list



$F^{(i+1)}(\cdot)$:

- Maintain a list of (x_j, y_j) . Let $S = \{y_j\}$
- If query is on the list, output y
- If not,
 - $i + 1$ times: sample y and add (x, y) to list
 - after: sample $y \notin S$ and add (x, y) to list



$F^{(i)}(\cdot)$:

- Maintain a list of (x_j, y_j) . Let $S = \{y_j\}$
- If query is on the list, output y
- If not,
 - i times: sample y and add (x, y) to list
 - after: sample $y \notin S$ and add (x, y) to list

Key point:

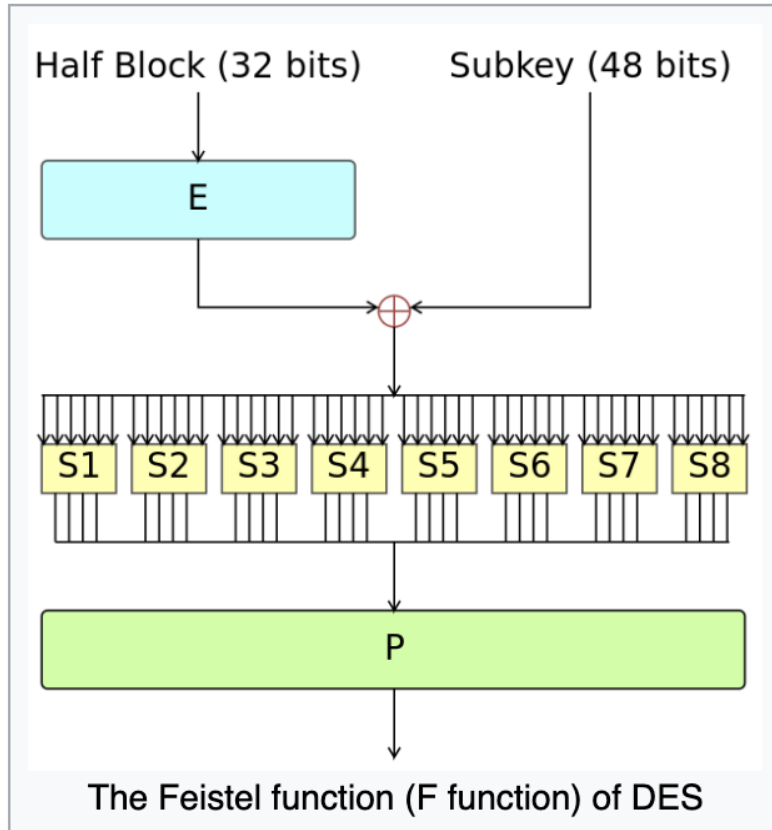
$$y \leftarrow \{0,1\}^n \approx_{\text{negl}} y \leftarrow \{0,1\}^n - S_i$$

$F^{(i+1)}(\cdot)$:

- Maintain a list of (x_j, y_j) . Let $S = \{y_j\}$
- If query is on the list, output y
- If not,
 - $i + 1$ times: sample y and add (x, y) to list
 - after: sample $y \notin S$ and add (x, y) to list

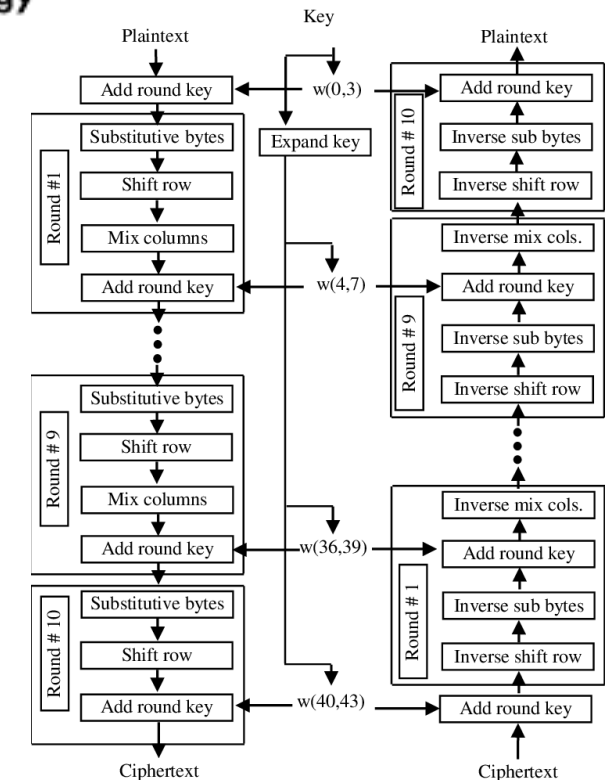
Why do we care about PRPs?

Data Encryption Standard



Real-world SKE
implementations

“Block ciphers”



Encryption from PRPs

Given a PRP, **several ways** to implement an encryption scheme (“modes of operation”). All involve breaking the message into “**blocks**” of length n . Low communication overhead!

- **ECB Mode:**

$$\text{Enc}(k, m_1, \dots, m_t) = P(k, m_1), P(k, m_2), \dots, P(k, m_t)$$

$$\text{Dec}(k, c_1, \dots, c_t) = P^{-1}(k, c_1), P^{-1}(k, c_2), \dots, P^{-1}(k, c_t)$$

Pros: very simple and length-preserving! **Cons:** deterministic, not CPA-secure

- **CBC Mode:**

$$\text{Enc}(k, m_1, \dots, m_t) = (\text{IV}, c_1 = P(k, \text{IV} \oplus m_1), c_2 = P(k, c_1 \oplus m_2), \dots, P(k, c_{t-1} \oplus m_t))$$

Pros: CPA-secure. **Cons:** highly sequential Enc (though Dec parallel)

- **CTR Mode:**

$$\text{Enc}(k, m_1, \dots, m_t) = (\text{IV}, P(k, \text{IV}) \oplus m_1, P(k, \text{IV} + 1) \oplus m_2, \dots, P(k, \text{IV} + t - 1) \oplus m_t)$$

Pros: CPA-secure and parallel! **Cons:** can't decrypt if you lose a block

Encryption from PRPs

Takeaways:

- **PRPs** are versatile; an invertible permutation is a useful off-the-shelf tool
- Informal intuition: **naturally-occurring permutations** are “more secure” (in terms of **concrete (T, ϵ)**) than functions with collisions.
- Important design paradigms naturally yield PRPs as a “base form” of cryptography, rather than a PRG or other kind of PRF.

Constructing PRPs

Constructing PRPs

1. Theorem (Luby-Rackoff): if PRFs exist, then PRPs exist!
 - “Feistel Networks”
 - Related to the (heuristic) construction of DES.
2. Substitution-Permutation Networks (SPNs)
 - (heuristic) construction of AES.

Constructing PRPs

1. Theorem (Luby-Rackoff): if PRFs exist, then PRPs exist!
 - “Feistel Networks”
 - Related to the (heuristic) construction of DES.

Constructing PRPs

Idea: “reversible computation”

$$\begin{array}{c} (x, y) \in \{0,1\}^{2n} \\ \downarrow \\ P_k \\ (x_1, y) = (x \oplus F_k(y), y) \end{array}$$

Is this secure?

Constructing PRPs

Idea: “iterated reversible computation”

$$(x, y) \in \{0,1\}^{2n}$$



$$P_{k_1, k_2} \quad (x_1, y) = (x \oplus F_{k_1}(y), y)$$



$$(x_1, y_2) = (x_1, y \oplus F_{k_2}(x_1))$$

Is this secure?

$$P_{k_1, k_2}(x, y) \oplus P_{k_1, k_2}(x', y) = (x \oplus x', \text{stuff})$$

Constructing PRPs

Idea: “iterated reversible computation”

$$(x, y) \in \{0,1\}^{2n}$$



P_{k_1, k_2, k_3}

$$(x_1, y) = (x \oplus F_{k_1}(y), y)$$



$$(x_1, y_2) = (x_1, y \oplus F_{k_2}(x_1))$$



$$(x_3, y_2) = (x_1 \oplus F_{k_3}(y_2), y_2)$$

Is this secure?

Constructing PRPs

Idea: “iterated reversible computation”

$$\begin{array}{c} (x, y) \in \{0,1\}^{2n} \\ \downarrow \\ P_{k_1, k_2, k_3} \quad (x_1, y) = (x \oplus F_{k_1}(y), y) \\ \downarrow \\ (x_1, y_2) = (x_1, y \oplus F_{k_2}(x_1)) \\ \downarrow \\ (x_3, y_2) = (x_1 \oplus F_{k_3}(y_2), y_2) \end{array}$$

Theorem [Luby-Rackoff]: if $F(\cdot, \cdot)$ is a PRF and k_1, k_2, k_3 are sampled i.i.d., then P_{k_1, k_2, k_3} is a PRP!

Next time: Authentication and CCA-security

What if



impersonates



?